

Java Backend Architecture

Interview Series

Chapter 17.7

Kubernetes Storage

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 17.7 — Kubernetes Storage

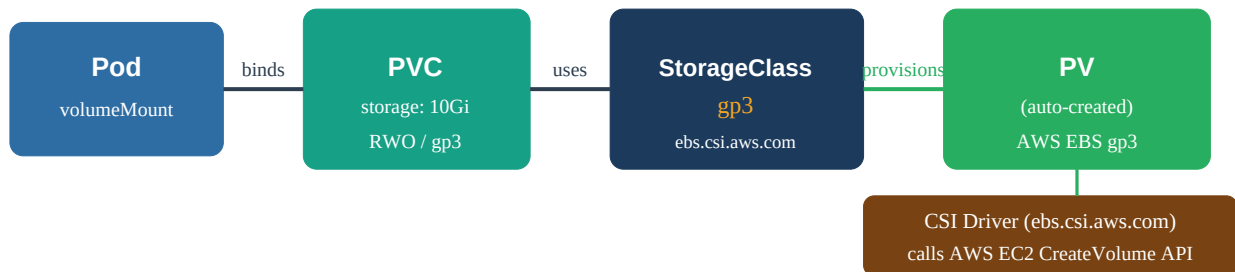
Overview

Kubernetes decouples storage from pods using PersistentVolumes (PV), PersistentVolumeClaims (PVC), and StorageClasses. Dynamic provisioning via CSI drivers automates volume lifecycle. StatefulSets pair with volumeClaimTemplates for stable, per-pod storage.

PersistentVolume & PersistentVolumeClaim

Dynamic Provisioning: PVC → StorageClass → PV (via CSI Driver)

reclaimPolicy: Delete (PV deleted with PVC) | Retain (PV kept for recovery)



- **PersistentVolume (PV)**: cluster-level resource representing actual storage (NFS mount, EBS volume, local disk)
- **PersistentVolumeClaim (PVC)**: user/pod's storage request — specifies size, access mode, storageClassName
- **Binding**: control plane matches PVC to PV (static) or creates PV on-demand (dynamic provisioning)
- **Phases**: Available → Bound → Released → (Reclaimed/Deleted)

```
# Static PV (manually created) apiVersion: v1 kind: PersistentVolume metadata: name: nfs-pv
spec: capacity: storage: 50Gi accessModes: [ReadWriteMany] persistentVolumeReclaimPolicy:
Retain nfs: server: nfs-server.example.com path: /exports/data --- # PVC requesting storage
apiVersion: v1 kind: PersistentVolumeClaim metadata: name: app-data spec: accessModes:
[ReadWriteOnce] storageClassName: gp3 resources: requests: storage: 10Gi --- # Pod using PVC
spec: volumes: - name: data persistentVolumeClaim: claimName: app-data containers: - name: app
volumeMounts: - name: data mountPath: /data
```

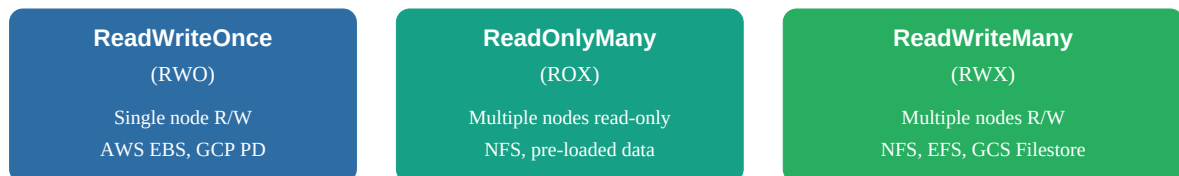
StorageClass & Dynamic Provisioning

- **provisioner**: ebs.csi.aws.com (AWS), pd.csi.storage.gke.io (GKE), disk.csi.azure.com (AKS)
- **parameters**: type (gp3/gp2/io1), fsType (ext4/xfs), iopsPerGB
- **reclaimPolicy: Delete** (default) — PV and cloud volume deleted when PVC deleted
- **reclaimPolicy: Retain** — PV kept after PVC deletion; must manually delete/reclaim
- **allowVolumeExpansion: true** — enables PVC resize (file system resize on pod restart)
- **volumeBindingMode: WaitForFirstConsumer** — provision in same AZ as pod (critical for AZ-local volumes)

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: gp3 annotations:
storageclass.kubernetes.io/is-default-class: "true" provisioner: ebs.csi.aws.com parameters:
type: gp3 fsType: ext4 throughput: "125" iops: "3000" reclaimPolicy: Delete
allowVolumeExpansion: true volumeBindingMode: WaitForFirstConsumer # AZ-aware provisioning
```

Access Modes

Access Modes Comparison



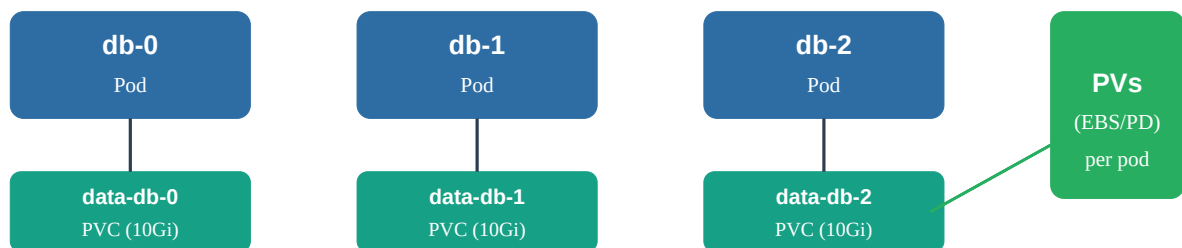
RWX required for shared storage across pod replicas (Deployment scale > 1)

RWO allows multiple pods on SAME node; most cloud block volumes are RWO only

Access Mode	Abbreviation	Multi-Node?	Common Backends
ReadWriteOnce	RWO	No (one node)	AWS EBS, GCP PD, Azure Disk
ReadOnlyMany	ROX	Yes (read only)	NFS, CephFS (read-only)
ReadWriteMany	RWX	Yes (read+write)	NFS, AWS EFS, GCP Filestore, CephFS
ReadWriteOncePod	RWOP	No (one pod only)	EBS CSI v1.x (Kubernetes 1.22+)

StatefulSet with volumeClaimTemplates

StatefulSet: Each Pod Gets Its Own PVC



volumeClaimTemplates auto-creates PVC per pod | PVC survives pod deletion

- Each StatefulSet pod (db-0, db-1, db-2) gets its own PVC from volumeClaimTemplates
- PVC name: <template-name>-<pod-name> — e.g., data-db-0
- PVC survives pod deletion — pod restart re-attaches same PVC (data persistence)
- Scaling down does NOT delete PVCs — must manually delete to reclaim storage

```
apiVersion: apps/v1 kind: StatefulSet metadata: name: db spec: serviceName: db replicas: 3
selector: matchLabels: app: db template: metadata: labels: app: db spec: containers: - name: db
image: postgres:15 volumeMounts: - name: data mountPath: /var/lib/postgresql/data
volumeClaimTemplates: - metadata: name: data spec: accessModes: [ReadWriteOnce]
storageClassName: gp3 resources: requests: storage: 10Gi
```

Volume Expansion

- Requires StorageClass allowVolumeExpansion: true
- Edit PVC spec.resources.requests.storage to new (larger) size
- CSI driver expands the block device; file system resize may require pod restart
- Online resize (no restart) supported by EBS CSI v0.10+, GCP CSI

```
# Expand PVC kubectl patch pvc app-data -p
'{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}' # Watch PVC condition kubectl get pvc
app-data -w # Condition: FileSystemResizePending -> FileSystemResizeSuccessful (after pod
restart if needed)
```

Volume Snapshots

- VolumeSnapshot CRD: point-in-time snapshot of PVC
- VolumeSnapshotClass: specifies CSI driver for snapshot
- Restore: create PVC with dataSource pointing to VolumeSnapshot

```
apiVersion: snapshot.storage.k8s.io/v1 kind: VolumeSnapshot metadata: name:
db-snapshot-20240101 spec: volumeSnapshotClassName: csi-aws-vsc source:
persistentVolumeClaimName: data-db-0 --- # Restore from snapshot spec: dataSource: name:
db-snapshot-20240101 kind: VolumeSnapshot apiGroup: snapshot.storage.k8s.io resources:
requests: storage: 10Gi
```

Local PVs for Performance

- Local PVs use node-local storage (NVMe SSD) — best IOPS/latency (etcd, Kafka, databases)
- No network overhead vs EBS/NFS
- nodeAffinity required — pod pinned to specific node
- Not suitable for HA — no replication; losing the node loses the data
- volumeBindingMode: WaitForFirstConsumer to match pod/node placement

Interview Q&A; — Kubernetes Storage

Q1. What is the difference between PV and PVC?

PersistentVolume (PV) is a cluster-level resource provisioned by admin or dynamically by CSI driver. It represents actual storage (EBS volume, NFS share, local disk). PersistentVolumeClaim (PVC) is a user's request for storage specifying size, access mode, and storageClassName. The control plane binds a PVC to a matching PV. PVC is namespaced; PV is cluster-scoped.

Q2. How does dynamic provisioning work?

1) Pod creates PVC referencing a StorageClass. 2) PVC controller sees unbound PVC with StorageClass. 3) CSI driver's external-provisioner watches for PVCs and calls the CSI CreateVolume RPC. 4) CSI driver calls cloud provider API (e.g., AWS CreateVolume). 5) CSI creates a PV and binds it to the PVC. 6) kubelet calls NodeStageVolume/NodePublishVolume to attach and mount on the node.

Q3. What is the difference between reclaimPolicy Retain and Delete?

Delete (default): when PVC is deleted, the PV is deleted AND the underlying cloud volume is deleted (data lost). Retain: PV moves to Released state; the cloud volume persists. An admin must manually recover data and delete/re-create the PV for reuse. Recycle (deprecated): basic scrub (rm -rf). Use Retain for production databases where accidental PVC deletion must not destroy data.

Q4. What is volumeBindingMode: WaitForFirstConsumer?

Default Immediate binding provisions the PV in any AZ, then schedules the pod. If the pod has nodeSelector or the node is in AZ-us-east-1a but the EBS volume is in us-east-1b, the pod cannot start (cross-AZ volume attachment not allowed for block storage). WaitForFirstConsumer delays PV provisioning until a pod using the PVC is scheduled to a node, then provisions in that node's AZ. Always use for zone-aware volumes.

Q5. What access mode would you use for shared storage across Deployment replicas?

ReadWriteMany (RWX). Block volumes (EBS, Azure Disk, GCP PD) only support RWO. For RWX, use: AWS EFS (NFS-based), GCP Filestore, Azure Files, NFS server, CephFS. Example: shared upload directory accessed by multiple API replicas uses EFS with RWX. Note: RWX doesn't mean concurrent write safety — applications must handle concurrency.

Q6. How does StatefulSet storage differ from Deployment storage?

Deployments: all replicas share one PVC (if using PVC in spec.volumes), or each pod gets the same emptyDir (ephemeral, per-pod). StatefulSets: volumeClaimTemplates creates one PVC per pod. pod-0 gets data-pod-0, pod-1 gets data-pod-1. These PVCs persist across pod restarts. Pod identity is stable — pod-0 always binds data-pod-0. This is essential for databases (Postgres, MySQL, Kafka, Cassandra).

Q7. What is the CSI driver architecture?

CSI (Container Storage Interface) decouples storage vendor code from k8s core. Components: external-provisioner (handles CreateVolume), external-attacher (ControllerPublishVolume), external-snapshotter, node-driver-registrar (NodeGetInfo, NodePublishVolume — runs on every node). Controller plugin runs as Deployment; Node plugin runs as DaemonSet. k8s kubelet calls CSI node service for mount/unmount operations.

Q8. How do you resize a PVC?

1) StorageClass must have allowVolumeExpansion: true. 2) `kubectl patch pvc -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'`. 3) CSI driver expands the block device (online if supported). 4) File system resize: if CSI supports online resize (EBS CSI v0.10+), it happens automatically. Otherwise, pod must be restarted to trigger fs resize. Note: you can only increase size, not decrease (no shrink support).

Q9. What happens to PVCs when a StatefulSet is scaled down?

PVCs are NOT deleted. The pods are removed but PVCs remain in Released state. When scaling back up, the same PVCs are re-bound to the new pods (by name convention). This prevents accidental data loss. To free storage, manually: `kubectl delete pvc data-db-2` (then the PV is reclaimed per `reclaimPolicy`).

Q10. What is the difference between emptyDir, hostPath, and PVC?

`emptyDir`: ephemeral volume shared among containers in a pod; data lost when pod terminates. Useful for scratch space, inter-container file sharing, `tmpfs` (memory-backed). `hostPath`: mounts a specific path from the host node. Security risk (access to node filesystem). Used for `DaemonSets` (Fluentd log dir, Docker socket). `PVC`: persistent storage that survives pod lifecycle, backed by real storage infrastructure.

Q11. What is a VolumeSnapshot and how do you use it for backup?

`VolumeSnapshot` creates a point-in-time copy of PVC data using storage provider's snapshot capability (EBS snapshot, GCP snapshot). Steps: create `VolumeSnapshotClass` (once, per CSI driver), create `VolumeSnapshot` referencing PVC. To restore: create new PVC with `spec.dataSource.kind: VolumeSnapshot`. Velero uses `VolumeSnapshots` for stateful app backup. Schedule regular `VolumeSnapshots` via `CronJob` or Velero schedule.

Q12. What is a local PV and when would you use it?

Local PV mounts a node-local disk directly (bypassing network I/O). Use for: `etcd` (requires low-latency storage), Kafka brokers, high-IOPS databases. Performance: NVMe SSDs provide 100k+ IOPS vs 3-16k IOPS for cloud block storage. Trade-off: no HA — if node fails, data is unavailable until node recovers. Requires `nodeAffinity` so pods are always scheduled to the node with that disk.

Q13. How do you handle storage for databases in Kubernetes?

1) Use `StatefulSet` with `volumeClaimTemplates`. 2) `StorageClass` with `gp3/io2` for OLTP, Retain reclaim policy. 3) Enable volume snapshots for backup. 4) Set `topologySpreadConstraints` to spread across AZs (for replicated DBs). 5) Anti-affinity to prevent multiple DB pods on same node. 6) Consider using managed databases (RDS, Cloud SQL) instead of running stateful workloads in k8s.

Q14. What is the projected volume use case?

A projected volume maps multiple source types (`Secret`, `ConfigMap`, `ServiceAccountToken`, `DownwardAPI`) into a single directory. Use case: application reads `/etc/config/db-password` (from `Secret`), `/etc/config/app.properties` (from `ConfigMap`), `/etc/config/token` (projected SA token) — all from one `mountPath`. Also used for short-lived SA tokens with custom expiry and audience (`TokenProjection`), required for OIDC-based auth (Vault, AWS IRSA).

Q15. What is the StorageClass default class?

Annotating a `StorageClass` with `storageclass.kubernetes.io/is-default-class: 'true'` makes it the default. PVCs without `storageClassName` specified use this default. Only one default `StorageClass` should exist per cluster (multiple defaults cause conflicts). In EKS, `gp2` is default; migrate to `gp3` by creating new `StorageClass` and updating default annotation.

Q16. What is ReadWriteOncePod (RWOP)?

Introduced in Kubernetes 1.22, RWOP ensures only ONE specific pod (not just one node) can write to the volume at a time. This is stricter than RWO where multiple pods on the same node could access the volume. RWOP prevents data corruption for single-writer workloads. Requires CSI driver support. Use for exclusive-access databases or lock files.

Q17. How does NFS storage work in Kubernetes?

NFS (Network File System) server runs outside k8s (or as a pod). Static PV: admin creates PV with `nfs.server` and `nfs.path`. Dynamic: NFS CSI driver or NFS subdir external provisioner creates subdirectories per PVC. NFS supports RWX — multiple pods can read and write. Performance: adequate for most file sharing, but higher latency than block storage. AWS EFS is managed NFS — use `aws-efs-csi-driver` with `StorageClass`.

Q18. What metrics do you monitor for Kubernetes storage?

`kubelet_volume_stats_used_bytes` and `kubelet_volume_stats_capacity_bytes` — PVC usage. Alert when usage > 85% capacity. CSI metrics: `csi_operations_seconds_bucket` — latency per operation. PV phase: watch for PVCs stuck in Pending (provisioning failure) or Lost state. `kubectl get pvc -A` — shows all PVCs and their binding status. Velero backup/restore metrics for data protection SLAs.

Q19. How do you migrate data between storage classes?

1) Create new PVC with target `StorageClass`. 2) Spin up a data migration pod (`rsync`) with both PVCs mounted. 3) `rsync /source/ /dest/`. 4) Update `Deployment/StatefulSet` to use new PVC. 5) Delete old PVC (verify `reclaimPolicy` before deletion!). Velero can also help: backup from source cluster, restore to target with different `StorageClass` mapping.

Q20. What is Rook/Ceph in Kubernetes?

Rook is a cloud-native storage orchestrator that deploys and manages Ceph (distributed storage) on k8s nodes. Provides: `CephBlockPool` (RWO block storage), `CephFilesystem` (CephFS RWX), `CephObjectStore` (S3-compatible). Useful for on-premise clusters needing software-defined storage. Cloud clusters typically use cloud-native CSI drivers (EBS, GCP PD) instead.

Q21. How do you handle PVC in Helm chart upgrades?

PVCs created by Helm are not updated on helm upgrade (immutable spec). If `StorageClass` or size needs to change, delete PVC and let it recreate (data loss risk!). Best practice: annotate PVC with `helm.sh/resource-policy: keep` to prevent Helm from deleting PVC on helm uninstall. For `StatefulSet` `volumeClaimTemplates`: Helm cannot update them — must delete `StatefulSet` manually (`--cascade=orphan` to keep pods) and re-apply.

Q22. What is the Container Storage Interface (CSI) migration status?

Kubernetes is deprecating in-tree storage plugins (e.g., `kubernetes.io/aws-ebs`) in favor of out-of-tree CSI drivers. Migration feature gates (`CSIMigrationAWS`, `CSIMigrationGCE`) redirect in-tree volume calls to CSI drivers transparently. By k8s 1.27, most in-tree plugins are removed. Always use CSI drivers: `ebs.csi.aws.com`, `pd.csi.storage.gke.io`, `disk.csi.azure.com`.

Q23. How does volume mounting work for multi-container pods?

Volumes are defined at pod level and mounted by individual containers. Multiple containers in a pod can share an `emptyDir` or PVC volume — useful for sidecar patterns. Example: main container writes logs to `/var/log` (`emptyDir`), `Fluentd` sidecar reads and ships logs. For PVC: both containers can mount the same PVC (subject to access mode restrictions — RWO allows multiple containers on same node).

Q24. What is the difference between Longhorn and Rook/Ceph for local cluster storage?

Longhorn (Rancher): simpler to install, built-in UI, replication across nodes, snapshot/backup to S3. Good for small-medium clusters and edge. Rook/Ceph: more complex, production-grade distributed storage (RADOS), supports block/file/object. Better for large-scale clusters needing all three storage types. Both provide dynamic provisioning via CSI. For cloud clusters, use cloud-native CSI drivers instead.

Q25. How do you prevent accidental PV deletion?

1) `reclaimPolicy: Retain` — PV/underlying volume not deleted when PVC is deleted. 2) `kubernetes` finalizers: PV has `kubernetes.io/pv-protection` finalizer — prevents deletion while bound. 3) RBAC: restrict 'delete' verb

on persistentvolumes to admins only. 4) Velero backup: regular PV snapshots for point-in-time recovery. 5) Never use reclaimPolicy: Delete for production databases.